

T2: First Iteration

 Published

 Assign To

 Edit



Team Assignment 2: First Iteration

This is a very dense assignment, you are responsible for reading this specification in its entirety.

Prerequisite

By the week following the due date of the project proposals your project mentor should have reached out to you indicating whether you are clear to proceed or whether you will need to meet to discuss revisions, if you need to revise your project proposal you will need to include it in your project's repository otherwise this step is not necessary. If you have not heard from your project mentor by October 1st please reach out to the Head IA for assistance.

Overview

Implement a demoable version of your service and make sure all the implementation code has been committed to the main branch of your github repository (it's ok to have some other code in branches but it will not be considered part of your submission). Your main branch MUST include the optional branch protection rules from the individual project + required review from at least one of your other team members. All main branch commit messages, except for the initial repo-setup commit(s), should say something meaningful about what was added/removed/changed. You may choose to follow a commit convention style but this is not required. This version of your service does not need to include all the functionality you proposed, but some API entry points must be operational in order to demo. Make sure to **tag**  (<https://stackoverflow.com/questions/18216991/create-a-tag-in-a-github-repository>) the set of file revisions considered part of the first iteration.

Style Checking + Internal Documentation Requirements

Your entire codebase, including test cases and even trivial code, should be compliant with a style checker appropriate for your language/platform.

Include some reports from your style checker in your repository. If it does not generate files, use redirection of output to a file, screenshots, or some other means to produce files.

All non-trivial code, including test cases, should either be self-documenting with mnemonic identifiers and header comments and, where appropriate, in-line comments or properly documented using an

appropriate tool for your language (e.g. javadoc for Java) You need not do this for trivial code. (Trivial means things like getters, setters, small helper functions, and other self-evident code.

There is no point in commenting `int addone (x) { return x+1 }.`

Testing Requirements

Implement and run unit tests for all the major "units" (methods, functions, procedures, subroutines, etc.) in your code. You should use a mocking framework if applicable. Unit tests should be grouped so related code (e.g., classes, modules, files, etc.) share setup, teardown and mocks when appropriate. Your entire collection of unit tests should run "push-button" via some test runner appropriate for your language and platform. Try to fix most of the bugs found by the tests.

Implement and run API (system) tests that exercise your service's initial API, e.g., using some API testing tool (Postman and Thunderclient are excellent options). Your tests should exercise the multiple clients and persistent data aspects of your service. Try to fix most of the bugs found by the tests.

Make sure all your test code and scripts have been committed to the main branch of your repository (its ok to have some other test code in branches but it will not be considered part of your submission).

Important Note: Your tests do not need to achieve high coverage (85%), yet, try to aim for the same percentage as was required in the individual mini project. The high coverage requirement will be part of the second iteration. You will also use a static analysis bug finder and continuous integration during the second iteration, as well as add integration testing and a sample client. You can do any or all of these during the first iteration if you want, but it is not required until the second iteration.

External Documentation:

Your repository's README.md should document all the operational entry points to your service and/or link to some external API documentation you wrote. Cover all the inputs and outputs, including status codes. If certain entry points must be called in certain orders or never called in certain orders, make sure to say so. Your README should also include instructions explaining how to build, run and test your service. All configuration files needed for build, run and test must be included in your repo.

If any third-party code is included in your codebase, your README should also document exactly which code this is, where it resides in your repository, and where you got it from (e.g., download url). However, in most cases you should use a package manager or build system to fetch third-party libraries as needed rather than bloating your codebase. Here is a reminder that you are NOT allowed to utilize AI in order to complete this assignment except as part of the functionality of the service itself - and only then if approved by the professor.

You will also be required to document the tasks completed by each team member via a project management software (JIRA and Trello offer free tiers but you may use another if you are familiar). You should choose one of these services and add all your members to the board as well as your project mentor.

Submission instructions:

One member of your team should submit a url pointing to your codebase repository. We encourage you to make your repository public, but if you prefer to keep it private then make sure the instructor and all the Mentors have collaborator access! This should normally be the same repository from your proposal, but we ask you to submit again for this assignment to make it easier to use Coursework's "SpeedGrader" tool.

If you prefer to have a copy of this document, you may access a PDF version in the Files > Team Assignments > Specs > T2.pdf

Points 75

Submitting a website url

Due	For	Available from	Until
Oct 18, 2024	Everyone	Sep 21, 2024 at 12am	Oct 20, 2024 at 11:59pm

+ Create Rubric

🔍 Find Rubric